

Completing the Chain: Verified Implementations of Hash-Based Signatures and Their Security

Manuel Barbosa¹, François Dupressoir², Rui Fernandes³, Andreas Hülsing^{4,5},
Matthias Meijers⁴ and Pierre-Yves Strub⁶

¹ University of Porto (FCUP) and INESC TEC, Porto, Portugal
mbb@fc.up.pt

² University of Bristol, Bristol, United Kingdom
f.dupressoir@bristol.ac.uk

³ Max Planck Institute for Security and Privacy, Bochum, Germany
ruipedrol6@protonmail.com

⁴ Eindhoven University of Technology, Eindhoven, Netherlands
research@mmeijers.com

⁵ SandboxAQ, Palo Alto, USA
andreas@huelising.net

⁶ PQShield SAS, Paris, France
pierre-yves.strub@pqshield.com

Abstract. We present the first formally verified implementation of a hash-based signature scheme that is linked to a machine-checked proof of security. Specifically, we provide reference implementations of XMSS and XMSS^{MT} written in JASMIN, targeting the AMD64 architecture. Beyond the implementations, we provide formal EASYCRYPT specifications of XMSS and XMSS^{MT}, transcribed from RFC 8391, and prove that our implementations adhere to these specifications. Furthermore, for XMSS, we give a machine-checked proof that our specification of RFC 8391 refines the abstract specification proven secure in EasyCrypt by Barbosa, Dupressoir, Grégoire, Hülsing, Meijers and Strub [CRYPTO’23]. In particular, we prove the security of our specification via a reduction, demonstrating that breaking our specification contradicts the [CRYPTO’23] result for our instantiation. Consequently, our implementation is not only functionally correct, but also adheres to a specification that is proven secure. The core technical challenge in our work resides in bridging low-level implementations of TREEHASH algorithms with high-level functional specifications used in the pre-existing formalization.

Keywords: XMSS, EasyCrypt, Jasmin, hash-based signatures, formal verification

1 Introduction

Hash-based signatures have been standardized as post-quantum secure alternatives to RSA- and discrete-logarithm-based signatures since 2018 when the first RFC on XMSS was published [HBG⁺18], followed by one for LMS [MCF19] shortly after. In 2020, the National Institute of Standards and Technology (NIST) published a standard [CAD⁺20] that also covers these two schemes, making them usable in settings with NIST-compliance requirements. These schemes are stateful, providing a solution for applications that can handle a secret state, i.e., a signing key that changes over time. More recently,

M.B. carried out part of this work while at the Max Planck Institute for Security and Privacy. F.D. is funded in part by Innovate UK SCHEME (Safety Critical Harsh Environment Micro-processing Evolution) project (10065634). A.H. and M.M. are supported by an NWO VIDI grant (Project No. VI.Vidi.193.066).

SPHINCS⁺ was standardized as SLH-DSA in FIPS-205 [Nat24]; as opposed to XMSS and LMS, SPHINCS⁺ is stateless, meaning it requires no management of secret state as it has a fixed signing key. The security of hash-based signature schemes relies *only* on computational assumptions about hash functions and *not* on the hardness of any other mathematical problems. This is contrary to most other types of signature schemes, for which security relies on both. For this reason, hash-based signature schemes are considered more likely to withstand potential advances in quantum cryptanalysis, and hence a more conservative choice for post-quantum digital signatures. However, for this same reason, hash-based signatures pose new challenges when it comes to proving security and optimizing implementations for performance.

HIGH-ASSURANCE CRYPTOGRAPHY. High-assurance cryptography [BBB⁺21] relies on formal methods to obtain machine-checked guarantees of security and correctness for cryptographic designs and implementations. For cryptographic designs—e.g., the new post-quantum cryptography standards—the relevant guarantee is whether the standardized specifications of the algorithms, written in pseudocode in the documents published by standardization bodies, satisfy a customary notion of cryptographic security such as IND-CCA (for KEMs) or EUF-CMA (for digital signatures). A proof of security that supports such claims must account for all concrete details in the standards document, which instantiate many loosely defined (abstract) aspects in the theoretical cryptography papers used to support the security of the standardized specifications. Indeed, the field of real-world cryptography [DPW11] is fertile in examples where choices made when writing a standard led to designs that are not actually covered by theoretical security proofs. For cryptographic implementations, there are several security-relevant goals that can be validated to a high degree of assurance with formal methods. Particularly, in increasing level of ambition, one can consider 1) memory safety and absence of undefined behavior due to, e.g., overflows; 2) correct deployment of countermeasures against timing and speculative execution attacks; and 3) functional correctness with respect to standard specifications. All these goals must be met to reach a high level of assurance about whether the security properties proved for standard specifications actually apply to the implementation. When it comes to post-quantum cryptography, this kind of end-to-end proof has only been done for ML-KEM implementations [ABB⁺23, ABB⁺19]. For the hash-based signature standards mentioned above, the cryptographic designs were also validated using computer-aided cryptography, starting in [BDG⁺23] with a machine-checked security proof of XMSS in EasyCrypt. This was later extended in [BDH⁺24] to a machine-checked security proof of SPHINCS⁺. Nevertheless, to the best of our knowledge, there are currently no results for formally verified implementations of hash-based signatures and, in particular, no results that formally connect implementations of such signatures to provably secure specifications.

CHALLENGES. At first sight it might seem that implementations of hash-based signatures would be an easier target for formal verification than, say, lattice-based constructions such as ML-KEM or ML-DSA. Certainly, hash-based signatures do not rely on some of the more advanced concepts that these lattice-based constructions rely on, such as polynomial rings, rejection sampling, and sampling from non-trivial distributions. However, in this work, we demonstrate that hash-based signatures introduce different and interesting challenges that render the proofs non-trivial. The main reason for this is the need to deal with tree-traversal algorithms implemented in an efficient, non-recursive way. Indeed, revisiting the security proof for XMSS given in [BDG⁺23], we find that the specification of the TREEHASH algorithm—central to the overall construction—is expressed as a recursive function over an inductive type for trees. However, both the relevant standards and reference implementation, which serve as guides for implementers, specify TREEHASH in a non-recursive way and make explicit use of a stack structure. Furthermore, the security proof builds on several cryptographic abstractions that are not explicit in the XMSS standards. For example, the proof relies on Tweakable Hash Functions (THFs), which are

abstractions of Keyed Hash Functions (KHF) that incorporate different levels of domain separation and randomization of hashing. THFs were only introduced with SPHINCS⁺, after the XMSS specifications in the above-mentioned standards (and implementations thereof) were written and published. This means that the XMSS specifications do not explicitly rely on THFs, but rather on standard (non-tweaked) hash functions. Similarly, the security proof abstracts WOTS⁺ public-key compression, treating it as a multi-target collision-resistant hash function, and the PseudoRandom Function family (PRF) enabling the compression of the XMSS secret key. In the XMSS standards, all these are concretely described using a bare-bones cryptographic hash function applied in different ways.

These abstraction gaps highlight the challenges that arise when carrying out machine-checked end-to-end security proofs, i.e., a chain of theorems that relates provable security results to low-level implementations. These challenges have been discussed in the literature [ABB⁺23, ABB⁺19, ABBD16, ABB⁺19, ABBD13], and the most challenging abstraction gap typically seems to occur between the cryptographic standard and its implementation. In this work, however, we encounter a different pattern. The reason for this is as follows. Concerning implementation correctness, our proof essentially establishes that the implementation carries out a sequence of calls to the underlying hash function that is consistent with that prescribed by the XMSS standards. Although this proof requires notable effort, it is relatively simple compared to other functional correctness proofs encountered in the literature, where the implementation of complex algebra needs to be considered. On the other hand, when we relate the XMSS standard to the security specification, we must deal with two different complex problems: 1) the functional correctness of the stack-based TREEHASH algorithm—this is essentially the functional correctness of a complex data structure implementation—and 2) matching the security semantics of various artifacts in the security proof (e.g., THFs, PRF, multi-target collision-resistant public-key compression) to different usages of the same hash function in the XMSS standard specification.

OUR CONTRIBUTIONS. This work addresses the gaps discussed above, presenting the first formally verified implementation of XMSS whose correctness and security is supported by a machine-checked proof. In particular, we make the following contributions:

- We give an EASYCRYPT specification of XMSS and its multi-tree variant, XMSS^{MT}, that is syntactically close enough to the specification from the RFC [HBG⁺18] to be easily checked for compliance by human inspection. Here, we further prove that XMSS is functionally equivalent to single-layer XMSS^{MT}.
- We prove that our EASYCRYPT specification of XMSS is functionally equivalent to a specific instantiation of the abstract specification used by Barbosa, Dupressoir, Grégoire, Hülsing, Meijers and Strub in their security proof for XMSS [BDG⁺23]. This represents the primary technical challenge, as it bridges the abstraction gap highlighted above. Based on this result, we also prove the security of the RFC specification of XMSS as a signature scheme via a reduction that relates it to the result in [BDG⁺23]. This proof clarifies under what conditions the previously established abstract security proof applies to the XMSS RFC specification and, in particular, it highlights under which conditions new parameter sets can be securely added (in potential extensions) to standards.
- We provide a reference implementation of XMSS^{MT} (and XMSS) in JASMIN, targeting the AMD64 architecture, that is closely aligned with the original C reference implementation that accompanies RFC 8391 [HBG⁺18], and its performance is comparable. Further, our implementation is verified for constant-time security using the tool that comes with the JASMIN compiler.
- We give an EASYCRYPT proof of the functional correctness of our JASMIN implementation of XMSS—i.e., single-layer XMSS^{MT}—with respect to the EASYCRYPT XMSS

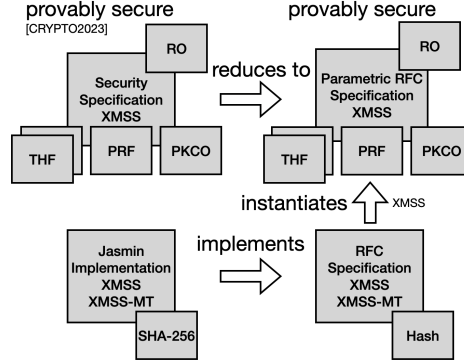


Figure 1: Chain of machine-checked results connecting implementation to security proof.

specification. This allows us to conclude that our implementation of XMSS is not only functionally correct, but also secure.¹

- Finally, the work in this paper is also a significant step towards the full-chain formal verification of SPHINCS⁺, given that SPHINCS⁺ uses XMSS^{MT} as a subroutine. However, due to a lot of differing details between the two schemes and the use of a few-time signature scheme in SPHINCS⁺ that is non-existent in XMSS, performing such a work for SPHINCS⁺ will still be significant effort.

THE CHAIN. Figure 1 depicts the relations between the chain of results we discuss in this paper. The bottom left shows the JASMIN implementations of XMSS and XMSS^{MT}, which are fully concrete and rely on the SHA-256 hash function as an external module. The top left shows the security specification given in [BDG⁺23], which we view as being parameterized by several low-level cryptographic primitives: the THFs used to compute hash chains and Merkle Trees; the Public-Key Compression function (PKCO) used to compress WOTS public keys to leaves; the PRF used to obtain pseudorandom strings from short seeds in various points of the computation; and a Random Oracle (RO) used to map messages to representatives. Under computational assumptions on these underlying primitives, [BDG⁺23] proved this specification secure in EASYCRYPT as a digital signature scheme in the Random Oracle Model (ROM). Then, the right-hand side of the diagram shows two versions of the formal specification of XMSS (and one of XMSS^{MT}). The bottom right shows our transcription of RFC 8391 [HBG⁺18], which we use to prove that our JASMIN implementation of the RFC is functionally correct and, therefore, compliant with the standard for a well-defined set of parameters. The top right shows a re-interpretation of the RFC specification that allows us to connect the RFC specification on the bottom right to the security specification on the top left: we start from the RFC specification and modularize the components that allow us to recover a parametric structure similar to the one used in the security proof. Intuitively, we obtain a specification that uses the security abstraction’s THF, PRF, PKCO and RO abstractions, but is otherwise identical to the RFC specification. This allows us to state, consistently with the security proof, which security assumptions are required over those abstractions to prove the RFC specification secure. As a sanity check, we prove that there is a natural instantiation of the specification on the top right that yields the specification on the bottom right, which means that our re-interpretation is merely a syntactic rearrangement. Finally, we prove that the specification on the top right is itself a secure digital signature scheme. We do this by converting any attacker against the specification on the right into an attacker against the security specification on the left. The most technically challenging proof is hidden

¹The multi-layer variant is not covered by the security proof of [BDG⁺23], and hence also not covered by the reduction given in this work.

inside this latter reduction: we need to prove functional equivalence between the abstract specification used in the security proof of [BDG⁺23] and that of the RFC [HBG⁺18].

STRUCTURE OF THE PAPER. The remainder of the paper is structured as follows. The upcoming section provides some background on JASMIN and EASYCRYPT, the tools used for the implementation and formal verification, respectively. Then, Section 3 introduces the XMSS and XMSS^{MT} signature schemes, as well as their formal specification in EASYCRYPT (the bottom right of Figure 1). Subsequently, Section 4 describes the implementation of XMSS^{MT} in JASMIN, presents benchmarks, compares our implementation to the existing C reference implementation, and discusses the proof of functional correctness (the bottom of Figure 1). Section 5 presents the set of results that connect the RFC specification to the security proof in [BDG⁺23] (the top and right of Figure 1). Finally, Section 6 concludes the paper, discussing the results and outlining directions for future work.

ACCESS TO DEVELOPMENT. The development is available at github.com/formosa-crypto/formosa-xmss.

2 Background on Jasmin & EasyCrypt

JASMIN. JASMIN [ABB⁺17, ABB⁺20] is a high-level assembly language designed for cryptographic implementations. The JASMIN compiler formally guarantees that the semantics [ABB⁺17] and constant-time properties [BGLP21, SBG⁺22] of safe source programs are preserved in the compiled programs. The language, often introduced as “assembly-in-the-head” provides high-level zero-cost abstractions such as variables and arrays, while giving the programmer control over low-level considerations—including instruction selection, and how loops should be compiled. Furthermore, it also provides a set of primitives to implement Selective Speculative Load Hardening (selSLH) and a type system to check the correct deployment of protection against Spectre v1 [SBG⁺23]. The compiler also supports mechanisms to zeroize the stack at the end of (export) functions [OBG⁺24], guaranteeing that all secret data is effectively erased from the stack upon function return. The applicability of JASMIN to the development of high-speed, high-assurance cryptographic primitives has been demonstrated most notably through its application to SHA-3 [ABB⁺19] and ML-KEM [ABB⁺23, AOB⁺24]. In both cases the JASMIN implementation was proven in EASYCRYPT to securely refine a specification whose security (indifferentiability from a random oracle for SHA-3; IND-CCA for ML-KEM) had also been formalized in EASYCRYPT.

EASYCRYPT. EASYCRYPT² [BDG⁺13] is an interactive proof assistant designed to formalize black-box, reduction-based proofs of exact security bounds in the code-based game-playing style. As of late, it has been most notably applied to provide formal assurance on proposed [HMS22] and standardized [BDG⁺23, ?, AOB⁺24, BDH⁺24] post-quantum algorithms. Over time, and with the development of JASMIN, EASYCRYPT has further evolved into a somewhat more general proof assistant for cryptographic reasoning. In particular, the core reasoning tool for game-playing cryptographic proofs—a logic for proving equivalences between probabilistic programs—is generally understood to be well suited to reason about the equivalence between algorithmic specifications and reference implementations, as well as between reference and optimized implementations. In this work we take advantage of the various logics available in EASYCRYPT to complete an end-to-end proof that includes functional correctness of assembly, and a sequence of equivalence results that connect this implementation to a security proof. These logics include a core Ambient Logic (AL), where in particular one chooses how to structure a proof using the other logics; Hoare Logic (HL) and Probabilistic Hoare Logic (PHL), which permit proving properties

²easycrypt.info

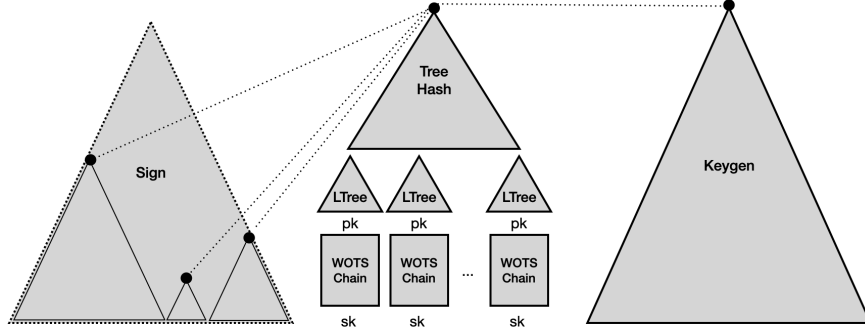


Figure 2: Reuse of TREEHASH in XMSS signing and key generation.

of single programs; and Probabilistic Relational Hoare Logic (PRHL) that permits proving equivalences between two programs.

TRUSTED COMPUTING BASE AND OTHER LIMITATIONS. We inherit from the JASMIN compiler a trust in the unverified OCaml code that handles parsing and typing, as well as the code that exports models of JASMIN programs to EASYCRYPT. The EASYCRYPT code base and SMT solver backends via Why3 are also part of our trusted computing base. As mentioned later in the paper, our proof does not cover the correctness of the implementation of the underlying SHA-256 implementation, which is the subject of concurrent work. A final caveat concerns safety verification. The static analyser included in the JASMIN compiler is not able to check the safety of our code, due to the intricacy of tree traversal algorithms. Although our functional correctness proofs, as described in Sections 4 and 5, require proving as auxiliary results that many accesses to arrays and memory are within bounds, we also use *valgrind* to check memory safety of our code. A mechanism for JASMIN safety verification is currently under development, which we plan to use once it is available.

3 The XMSS RFC and its Formal Specification

This section describes the XMSS and XMSS^{MT} signature schemes and their formalization in EASYCRYPT following RFC 8391 [HBG⁺18]. Rather than including large code snippets in this document, we include links to an external repository that we use as supplementary material. XMSS [BDH11] is a stateful hash-based signature scheme that combines a one-time signature scheme—WOTS⁺—and a Merkle tree of height h . This is used to authenticate each of the 2^h WOTS⁺ public keys, which correspond to the leaves of the binary hash tree. Each signature “consumes” one WOTS⁺ key and the signer updates the secret key by incrementing a counter that ensures each WOTS⁺ key is used only once. XMSS^{MT} generalizes XMSS by replacing the single Merkle tree with a hypertree of d layers, each of which contains a Merkle tree of height h/d : the topmost tree is used to authenticate the roots of the trees in the level below, and this pattern is then followed in subsequent levels; the trees on the bottom layer are used to sign actual messages. This allows the total number of signatures to be increased while keeping running times manageable, at the expense of increased signature size.

MERKLE TREE, TREE HASH & AUTHENTICATION PATH. Each leaf of the tree corresponds to a (digest of a) WOTS⁺ public key—we explain WOTS⁺ works below. The root of the Merkle tree serves as the public key for XMSS. Intermediate nodes in the tree can be computed by applying a randomized hash function to its children. In particular, one computes the resulting node by XOR-ing the two child nodes with a pseudorandomly generated bitmask and then hashing these values together. The core algorithm in XMSS is called TREEHASH: it is used to compute the root of a subtree. Conceptually, TREEHASH

works by processing the leaves of the tree sequentially, using a stack to store intermediate nodes. On each iteration, the next leaf is pushed to the stack. Then, an internal loop reduces the stack as much as possible: when two nodes at the top of the stack have the same height, they are hashed together to form a node at the next (higher) level. This new node is then pushed onto the stack, replacing the two nodes that were hashed together. This process continues until all leaves in the subtree are processed. At that point, the stack contains a single element, which corresponds to the root of the subtree and is the final result. The EASYCRYPT specification of the TREEHASH algorithm, transcribed from RFC 8391 [HBG⁺18] can be found [here](#) [↗](#). This algorithm is the main component in key generation. It is also a core routine in the signature process, as shown in Figure 2. An XMSS signature comprises a WOTS⁺ signature and a *proof* of validity of the WOTS⁺ public key for that signature: the list of tree nodes (called authentication path) that permits reconstructing the Merkle tree root from the WOTS⁺ public key. This can be constructed by iteratively computing the root of each subtree in the XMSS tree, starting from the leaves and up to the root. During signature generation, this path is computed by a sequence of calls to the TREEHASH algorithm above, as illustrated [here](#) [↗](#).

WOTS⁺. The one-time signature scheme WOTS⁺ is one of the building blocks of XMSS. This scheme is parameterized by the Winternitz parameter w and the security parameter n , that corresponds to the output length, in bytes, of the underlying hash function. It permits signing messages n -byte messages. From these, we derive additional parameters

$$\ell_1 = \left\lceil \frac{8 \cdot n}{\log_2 w} \right\rceil \quad \ell_2 = \lfloor \log_w(\ell_1 \cdot (w - 1)) \rfloor + 1$$

where ℓ_1 represents the number of base- w digits required to represent an n -byte integer value and ℓ_2 the number of base- w digits required to represent a checksum in the range $[0, \ell_1 \cdot (w - 1)]$. Finally, we define $\ell = \ell_1 + \ell_2$, that corresponds to the number of n -byte string elements in WOTS⁺ keys and signatures. Indeed, the WOTS⁺ secret key is simply a pseudorandom string of length $\ell \cdot n$ bytes, and it can be expanded from a n -byte secret seed by applying a PRF to distinct values—this is the norm for XMSS at least. The corresponding public key is then computed by applying a chaining function $w - 1$ times to each value of the secret key. The chaining function is defined as $c_k^i(x, \mathbf{r}) = f_k(c_k^{i-1}(x, \mathbf{r}) \oplus r_i)$; $c_k^0(x, \mathbf{r}) = x$, where k is a keying parameter for the hash function family and $\mathbf{r}$ are randomizers, which must be selected for each WOTS⁺ public key. (In XMSS these values are expanded from a global seed `pub_seed` using a PRF.)

Within XMSS, we only sign fixed-length messages of length n . To do so, a message is first encoded into a representation in base- w ³. This representation contains the base- w value of the message when seen as an integer and also a checksum (also represented in base- w) that guarantees that this encoding is cover-free.⁴ In total, ℓ values b_i in base- w result from this encoding procedure. The signature itself is then the result of applying $c_k^{b_i}(sk_i, \mathbf{r})$ to each element of the secret key. To verify a signature, it therefore suffices to confirm that completing the chains permits reconstructing the WOTS⁺ public key. [This code](#) [↗](#) shows how the WOTS⁺ signature type and algorithm are specified in EASYCRYPT. Each WOTS⁺ public key consists of ℓ n -byte elements. However, each leaf of the XMSS tree is only n bytes long. To compress a WOTS⁺ public key into a single n -byte value, XMSS uses the LTREE algorithm. LTREE builds yet another binary hash tree where each of the ℓ leaves correspond to one n -byte element of the WOTS⁺ public key. The root node is then the compressed representation of the WOTS⁺ public key and is used as a leaf of the XMSS tree. The EASYCRYPT specification of LTREE can be found [here](#) [↗](#).

³All resulting values are in the range $[0, w)$, and w can be chosen to set a trade-off between time and memory requirements for the signature.

⁴No encoding has all values greater equal than any other encoding which is required for security.

XMSS KEY GENERATION. ([EASYCRYPT code 4](#)) During key generation, one starts by sampling three n -byte random values. To have a shorter key size, XMSS uses a secret seed – `sk_seed` in the code—from which we can generate the WOTS^+ secret key via a PRF. The value `sk_prf` is used as a key to another PRF that produces randomizers for message hashing during signature generation. The public seed `pub_seed`, as mentioned above, is used as key to a PRF when generating bitmasks for randomized hashing. The index of the signing key is initially set to zero, and the public key is found by computing the root of the XMSS tree. The process is quite similar for both XMSS and XMSS^{MT} . The only difference is that, in XMSS^{MT} , the root of the top tree in the hyper tree (at height h/d) is computed.

XMSS SIGNING. ([EASYCRYPT code 4](#)) An XMSS signature combines a WOTS^+ signature and a Merkle tree authentication path that authenticates the one-time WOTS^+ public key. For XMSS^{MT} , we have a list of WOTS^+ signatures and corresponding authentication paths, one for each layer of the hypertree. The signer sets the index of the signature as the current index from the secret key—this index indicates which one-time key was used. The index in the secret key is then updated to prevent key reuse. Then, a pseudorandom value R is derived using a PRF and `sk_prf`, a message digest is computed (using R), and then signed using WOTS^+ . Finally, the authentication path for the leaf is computed. The signature includes the index of the WOTS^+ public key, the pseudorandom value R , the WOTS^+ signature, and the authentication path. This is illustrated [here 4](#). For the multi-tree variant, this procedure is repeated for each tree layer $j \in [1, d - 1]$, skipping the initial message digest.

XMSS VERIFICATION. ([EASYCRYPT code 4](#)) During signature verification, to check whether a given signature is valid, the verifier reconstructs the root of the XMSS tree from the WOTS^+ signature and the authentication path present in the XMSS signature. First, the message digest is recomputed using R . Then the WOTS^+ public key is computed from the WOTS^+ signature, the message digest, and the public seed by completing the hash chains. Next, the verifier uses the authentication path to compute the root of the XMSS tree. Recall that intermediate nodes of the tree are computed by hashing the two child nodes together. The authentication path contains all of the necessary nodes, allowing the verifier to compute the root of the XMSS tree. Finally, the verifier compares the reconstructed root with the root stored in the public key, and the signature is valid only if these match. In the XMSS^{MT} variant, the verification algorithm is iterated for each layer.

XMSS vs XMSS^{MT} . As a sanity check for the consistency of our specifications, we prove in EASYCRYPT that XMSS and XMSS^{MT} are functionally equivalent when the multi-tree version is instantiated with only one layer ($d = 1$). This step is also important for a proof management reason. It allows us to start from an implementation of XMSS^{MT} , which we prove functionally correct with respect to the XMSS^{MT} specification. For now, this is a more general result than we can connect to a security proof, which so far only exists in EASYCRYPT for XMSS. However by establishing that XMSS is simply a particular case of XMSS^{MT} for $d = 1$, this implies that our implementation of XMSS^{MT} for $d = 1$ falls under the scope of the security result for XMSS that we discuss in subsequent sections of this paper. The proof is formalized as a program equivalence in EASYCRYPT. [Figure 3](#) shows the lemma that captures this equivalence for the signature generation. This proof is straightforward. In the multi-tree variant, we treat the first layer (layer 0) separately. The hypertree traversal loop then begins at the second layer (layer 1), iterating until the last layer is reached. In addition, we have that, when $d = 1$, this loop is not executed. Thus, only the computations of the first layer are performed. As such, these two schemes are structurally identical in the sense that both perform the same sequence of operations. Similar results have been proven for key generation and signature verification.

```

equiv sign :
  XMSS_PRF.sign ~ XMSS_MT_PRF.sign :
    XMSS_Params.impl_oid = XMSS_MT_Params.impl_oid /\ d = 1 /\ h < 32
    /\ 0 < to_uint sk{2}.`idx < 2`h - 1 /\ arg{1}.`1 = arg{2}.`1 /\ arg{1}.`2 = arg{2}.`2
    ==> eqsig res{1}.`1 res{2}.`1 /\ res{1}.`2 = res{2}.`2.

```

Figure 3: Equivalence lemma between the signing algorithms of XMSS and XMSS^{MT}. A relational pre-condition sets the restricted context for the equivalence; the post condition guarantees identical outputs. Here, `._`_` denotes access to tuple/record components.

4 Implementation and Functional Correctness Proof

4.1 Implementing XMSS and XMSS^{MT} in Jasmin

This section briefly describes the implementation of the XMSS^{MT} signature scheme in JASMIN targeting the AMD64 architecture. The implementation is based on the C reference implementation⁵ provided by the authors of the scheme. The JASMIN implementation mirrors the C reference implementation’s structure for the most part. Both implementations favor readability and closeness to the specification, which means that they do not include optimizations that could improve performance. The C reference implementation uses OpenSSL’s implementation of SHA2, whereas our implementation uses the JASMIN implementation of SHA2 provided by libjade.⁶

OVERVIEW. As mentioned in Section 3, the EASYCRYPT specification of XMSS uses high-level constructs such as record types to represent structures like cryptographic keys and signatures. This allows for simpler reasoning when formalizing the correctness and security proof. However, when implementing the scheme, these structures must be mapped into the appropriate low-level representations that are compatible with the JASMIN programming language. In this case, integers are represented either as 32 or 64-bit machine words. Constants and values known at compile-time can be represented by `inline int`. Variables of this type are replaced by their respective values during compilation. In JASMIN, the size of memory/stack arrays must be known at compile time. When this is not the case—for example, we do not know the size of the message to sign at compile time—it is not possible to represent this data using JASMIN’s array types and the convenient array notation for accessing the array contents. Instead, we use a 64-bit machine word to store memory addresses (a raw pointer). Access to the elements of these buffers is performed explicitly via load and store operations through the memory address stored in a 64-bit register.

JASMIN IMPLEMENTATION. The fact that JASMIN arrays can only be used when their size is statically known has some implications, in particular it leads to some code duplication. For example, in our implementation, we need to have two different kinds of SHA2 implementation: 1) an implementation that hashes inputs that live in external memory and whose length is not known at compile time, which is done by directly manipulating memory addresses stored in 64-bit machine words; and 2) an implementation that hashes inputs that live on the JASMIN stack and whose input length is known at compile time.

In addition, in JASMIN, and in contrast to C, when a function takes as argument an array, its size must be specified. This means that, when writing a function that hashes its input, the size of the input must be fixed, and if we want to hash inputs of different lengths, then we need to write different functions, one for each input length.

Another aspect worth mentioning about JASMIN is that, if a function takes as argument more than one array, then these are assumed to be disjoint, i.e. they do not overlap. While this makes it easier to reason about function behaviour, especially when doing correctness

⁵github.com/XMSS/xmss-reference

⁶github.com/formosa-crypto/libjade

proofs, it needs to be taken into account when implementing a JASMIN program based on its C counterpart. For example, in the C reference implementation, the chaining function takes as arguments pointers to an input buffer and an output buffer. In C, it is possible that the same buffer is passed as both input and output, i.e. the input and output pointers point to the same memory location. In JASMIN, however, this is not possible. In this case, we are faced with the following options: 1) We can merge these two arguments into a single one, but in this case we are limited to the case where the input and output buffers point to the same memory location—which is the case in our implementation; 2) we can keep the two arguments separate, but in this case we need to ensure that they are disjoint, which means we may need a memory copy before calling the function if this is not the case; or 3) we can write two different functions, one for the case where the input and output buffers are disjoint, and another one for the case where they are not. These examples illustrate how implementing programs in JASMIN exposes the programmer to low-level details and subtle assumptions that are typically abstracted away in higher-level languages like C. The benefit, however, is that the resulting functions can be as fast as their C counterparts, while giving rise to formally proven correct assembly.

TREEHASH & COMPUTATION OF THE AUTHENTICATION PATH. The TREEHASH algorithm is used to compute the root of a subtree as discussed in the previous section. The JASMIN implementation of TREEHASH and its C counterpart are quite similar. Both use a stack to store the intermediate nodes of the tree, and iteratively loop through the nodes of the tree, pushing them onto the stack. Whenever the two topmost nodes of the stack are at the same level of tree, they are hashed together to compute their parent node, which is then pushed onto the stack, replacing these two nodes. This process is carried out until all of the leaves of the subtree have been processed, at which point the root of the subtree is the topmost node of the stack. The JASMIN implementation of TREEHASH is shown [here](#) [↗](#). To compute an authentication path, the JASMIN implementation follows the RFC and calls the TREEHASH algorithm for the subtree that gives rise to each node in the path. In contrast, the C reference implementation introduces an optimization that merges into TREEHASH the possibility to eagerly compute an authentication path as the Merkle tree root is computed. This deviates somewhat from the RFC specification, which goes against our goal that the specification and reference implementations in EASYCRYPT should allow an easy syntactic match to the RFC. We leave it for future work to deal with optimizations of TREEHASH and XMSS as a whole, which is a very interesting line of research.

CONSTANT-TIME SECURITY AND ZEROIZATION. We validate that our implementation is constant-time using JASMIN’s constant-time checker. This requires adding some annotations to the source code to guide the checker. In particular, the checker conservatively considers values loaded from memory as secret. However, this is not always the case. When we load public values from memory (e.g. the size of the message to sign) we tag them as public. Additionally, the JASMIN compiler supports zeroing the stack when returning from export functions – functions that can be called from external code. This ensures that no secret data is left on the stack after function execution. We evaluate two different zeroization strategies below: `loop` and `unrolled`. The first one uses a loop to zeroize the stack, while the second one unrolls the loop. Both use a series of `mov` instructions to zeroize the stack by writing a 64-bit word at a time. The optimal zeroization strategy depends on the specific scenario: the `loop` strategy reduces code size, while the `unroll` eliminates loop overhead, potentially offering better performance at the cost of an increased code size.

BENCHMARKING. We evaluate the performance of our JASMIN implementation—with and without zeroization enabled—by comparing it with the C reference implementation. Data was collected on a `x86_64` machine, with a Intel Core i7-6700 CPU, clocked at 3.40GHz, using a single core with both Hyperthreading and TurboBoost disabled to ensure consistent measurements. These results are shown in Table 1 and refer to the

Table 1: Performance Comparison between C and JASMIN implementations in 10^6 clock cycles (column #2). Zeroization overhead wrt JASMIN baseline (columns #3 and #4).

	C Ref	Jasmin (abs, %)	loop zeroization (abs, %)	unrolled zeroization (abs, %)
Keygen	10092.74	8263.54 (-18.12%)	8293.61 (+0.36%)	8312.35 (+0.59%)
Sign	20251.01	24775.17 (+22.34%)	24872.05 (+0.39%)	24927.53 (+0.61%)
Verify	10.35	8.47 (-18.14%)	8.48 (+0.13%)	8.95 (+5.69%)

Table 2: Comparison of time spent computing SHA2 2 hashes (in 10^6 clock cycles).

	C Ref	C ref using Jasmin SHA-2	Jasmin baseline
Keygen	9781.09	8000.87	7986.50
Sign	19573.38	16,011.18	23954.75
Verify	10.07	8.36	8.33

XMSSMT-SHA2_20/2_256 parameter set—i.e. XMSS^{MT} instantiated with $d = 2$ and $h = 20$, and using SHA2 as the underlying hash function. The results show that our JASMIN implementation is slightly faster than the C reference implementation, for key generation and verification, even though the implementations are structurally identical. This difference in performance can be attributed to the fact that our implementation of SHA2 is slightly faster than the OpenSSL SHA2 implementation used by the C reference implementation. In the case of the signature generation, the slightly worse performance of our implementation is due to the way in which we compute the authentication path, following the RFC, as discussed above. We now discuss these points more in details.

Table 2 illustrates the impact of the hash function implementation on the reference implementation, compared to the Jasmin baseline. It reports the total number of clock cycles that each implementation spends computing hashes: columns #1 and #3 are the reference and Jasmin implementations, respectively, whereas column #2 corresponds to a version of the C reference code where hash computations using OpenSSL have been replaced with the Jasmin SHA2 implementation. Comparing these results with the results of Table 1, it is clear that hash computation make up the majority of execution time. As a result, even slight improvements to the hash function implementation yield considerable improvements in overall performance. Indeed, when we replace OpenSSL’s SHA2 with the JASMIN implementation of SHA2 in the C reference code, we observe that we spend approximately 18% less time computing hashes, for key and signature generation, and 17% less on signature verification.

The other factor that affects performance of the analysed implementations is how the authentication path is computed during signature generation. In the C reference implementation, TREEHASH computes the root of a subtree and the authentication path simultaneously. On the other hand, the JASMIN implementation follows an approach similar to that of RFC 8391 [HBG⁺18], where TREEHASH computes only the root of a subtree. Authentication paths are then obtained by iteratively calling TREEHASH for each subtree level. This approach causes signature verification to be slower on our JASMIN implementation compared to the C reference implementation, which illustrates the impact of high-level algorithmic choices on the performance of hash-based signatures. Conversely, during key generation, the C reference implementation computes both the authentication path and the root of the tree, when only the latter is needed. On the JASMIN implementation, however this unnecessary computation is not performed. Regardless, this difference is not meaningful in terms of performance – since the C reference implementation copies tree nodes to the authentication path during root computation, these differences amount to only a few memory copy operations that do not significantly impact performance.

We see these results as a feasibility result for formal verification in a new family of

```

axiom hash_128 (x : W8.t Array128.t) :
  phoare[ M(Syscall).hash_plen_3n___sha256 : arg.`2 = x ==>
    to_list res = NBytes.val (Hash (to_list x)) ] = 1%r.

```

Figure 4: EASYCRYPT axiom stating the correctness of SHA2.

cryptographic algorithms: using our approach we are able to provide formally verified implementations that are on a par with unverified code when it comes to performance. Indeed, in terms of performance, our JASMIN implementation can play the same role as a C reference implementation, with the advantage of being formally verified. However, portability remains a limitation—our implementation only targets the AMD64 architecture.

As a final curiosity, we also report in Table 1 the performance of the JASMIN implementations considering different zeroization strategies: column #3 shows zeroization using a loop over the whole stack region, whereas column #4 uses unrolled code to perform the same operation. Interestingly, although both approaches have a negligible impact on the performance of the implementation, the unrolled version is slightly slower, which is counterintuitive. We attribute this to the increased size of the unrolled zeroization code (30 Kbyte overhead), which in turn indicates that the Jasmin program needs to clear up a pretty large region in the stack. Since the programmer has no control of the stack in the Jasmin language, we leave it as an interesting direction for future work to investigate potential improvements to the language and compiler that permit controlling the stack size.

4.2 Functional Correctness Proof

We prove the functional correctness of our XMSS^{MT} implementation written in JASMIN with respect to the RFC-like EASYCRYPT specification from Section 3. This proof relies on the JASMIN compiler to extract an EASYCRYPT representation of the JASMIN program, which is then used to establish the functional correctness of the implementation with respect to the specification. We prove functional correctness of the JASMIN code by proving PRLH equivalence statements that relate the extracted EASYCRYPT representation to the EASYCRYPT specification.

CORRECTNESS OF SHA-2 CODE. In the correctness proof, we do not prove the correctness of the underlying SHA2 hash function, which is the subject of ongoing independent work; instead, we assume that it produces the correct output: concretely, the semantics of the hash function implementation is axiomatized as shown in Figure 4, via a total correctness Hoare triple. The axiom establishes that, for byte arrays of size 128, the `hash_plen_3n___sha256` function—which originates from the JASMIN implementation—terminates with probability 1, and produces (upon termination) the same output as the abstract operator `Hash` used in the RFC specification.

ENCODING & DECODING. To express equivalences between the record-based EASYCRYPT specification and the array-based JASMIN implementations we need to define operators that relate the two representations. For example, the lemma `xmss_kg_correct` in Figure 5 establishes the correctness of the key generation algorithm. Essentially, it states that if the key generation algorithm is executed, then the resulting public key and private key can be decoded to match values produced by the specification. Here, the `DecodePkNoOID` and the `DecodeSkNoOID` operators are used to relate the representation of the cryptographic keys in the specification (represented as record types), and in the JASMIN implementation, where they are represented as byte arrays.⁷

⁷In the above, the `NoOID` suffix highlights that the implementation representation of the public keys does not include object identifier meta-information.

```

lemma xmss_kg_correct :
  n = XMSS_N /\ (* ... *) /\ F_padding_val = XMSS_HASH_PADDING_F =>
  equiv [ M(Syscall).__xmssmt_core_keypair ~ XMSS_MT_PRF.kg :
    true ==> res{1}.`1 = DecodePkNoOID res{2}.`2 /\ res{1}.`2 = DecodeSkNoOID res{2}.`1 ].

```

Figure 5: EASYCRYPT lemma for correctness of XMSS’s key generation algorithm.

```

pred valid_ptr_i (p : W64.t, o : int) =
  0 <= o => 0 <= to_uint p /\ to_uint p + o < W64.modulus.

pred disjoint_ptr (p1 l1 p2 l2 : int) =
  forall (k1 : int), 0 <= k1 < l1 => forall (k2 : int), 0 <= k2 < l2 => p1 + k1 <> p2 + k2.

```

Figure 6: Pointer predicates in EASYCRYPT.

```

lemma verify_correctness (ptr_m ptr_mlen ptr_sm sm_len : W64.t) (_pk : W8.t Array64.t) :
  n = XMSS_N /\ d = XMSS_D /\ ... /\ H_msg_padding_val = XMSS_HASH_PADDING_HASH =>
  equiv [
    M(Syscall).__xmssmt_core_sign_open ~ XMSS_MT_PRF.verify :
      disjoint_ptr (to_uint ptr_m) (to_uint sm_len) (to_uint ptr_mlen) 8 /\
      disjoint_ptr (to_uint ptr_sm) (to_uint sm_len) (to_uint ptr_m) (to_uint sm_len) /\
      disjoint_ptr (to_uint ptr_sm) (to_uint sm_len) (to_uint ptr_mlen) 8 /\ ...
    arg{1} = (ptr_m, ptr_mlen, ptr_sm, sm_len, _pk) /\
    arg{2}.`1 = EncodePkNoOID _pk /\
    arg{2}.`2 = load_buf Glob.mem{1} (ptr_sm + (of_int XMSS_SIG_BYTES)%W64)
              (to_uint (sm_len - (of_int XMSS_SIG_BYTES)%W64)) /\
    arg{2}.`3 = EncodeSignature (load_sig_mem Glob.mem{1} ptr_sm)
    ==> (res{2} <=> res{1} = W64.zero) ].

```

Figure 7: EASYCRYPT lemma for correctness of XMSS’s verification algorithm. Note that verification in the code is carried out by a function called `sign_open`, which returns the verification result bit encoded in a 64-bit word.

POINTERS. As mentioned in Section 4.1, it is not always possible to represent data using arrays. When this is not the case, we use a 64-bit machine word to store memory addresses. This approach requires us to reason explicitly about pointer arithmetic in the proofs. Since we cannot use JASMIN’s array types, we must explicitly add to our lemmas the assumption that the pointers are valid and that they are disjoint. To do so, we define two predicates, illustrated in Figure 6. In practical terms, `valid_ptr_i` states that accessing a pointer p with a non-negative offset o must not overflow the 64-bit address space, which is essential to guarantee that any memory read or write falls within a valid, allocated memory region. In addition, `disjoint_ptr` states that two memory regions starting at addresses p_1 and p_2 , of lengths l_1 and l_2 , respectively, do not overlap. As an example of the equivalence results we prove using relational Hoare logic, we show in Figure 7 the lemma that establishes the correctness of the signature verification function⁸ by establishing that the returned 64-bit result is zero if and only if verification succeeds. The proofs themselves are reasonably straightforward as EASYCRYPT proofs go, as they amount to establishing that implementation and specification are placing consistent calls to the underlying hash function. We give more detail about this for the most intricate proof (TREEHASH) below.

TREEHASH CORRECTNESS. The correctness proof of the (TREEHASH) algorithm was the

⁸Here, the disjointness assumptions guarantee that write operations to the output buffers `ptr_m` and `ptr_mlen` where the verify operation places the message that was validated by the signature do not interfere with each-other and also with the input buffer `ptr_sm` containing the signature and the signed message. It is possible that they can be relaxed, albeit not fully removed, at the expense of more fine-grained reasoning in the proof wrt when a write operation may potentially modify a value that is needed by a future computation.

most demanding part of the functional correctness proof, and in some places it would imply reproving some of the results about the behavior of the stack that we need to establish for the security proof result we will discuss in the next section. To go around this, we expressed the intricate functional correctness loop invariant for TREEHASH in a way that avoids this redundancy: rather than proving that implementation and specification are accessing the stack within bounds—which we discuss below for the specification—we prove that they perform identical accesses—which could be within bounds or out of bounds—and derive that they produce the same result. We note that considering out of bounds accesses is done for formal verification purposes, and that out of bounds accesses are never actually performed by the implementation: this is guaranteed by the fact that we also check the program for safety. Furthermore, and as mentioned above, we establish the stronger result that all accesses are within bounds for the RFC specification when we relate it to the security proof below. This implies by transitivity (although we do not prove it formally) that the accesses performed by the implementation are also within bounds.

5 Extending the Chain to the Security Proof

We now refer back to Figure 1 and focus on our use of the pre-existing EASYCRYPT results from [BDG⁺23]. Concretely, we first describe how we instantiate some of the abstract components in the security proof of XMSS to derive a security theorem for a refined description of the scheme that we, in turn, match to the concrete specification from the XMSS RFC. Later in this section we discuss how we obtain a security result that applies to the RFC via a security reduction.

5.1 Instantiating the Security Proof From [BDG⁺23]

The security result for XMSS in [BDG⁺23] can be summarized by the following three equations, whose meaning we describe only at the high-level, omitting some statistical terms not relevant to our discussion.

$$\begin{aligned}\epsilon_{\text{XMSS}}^{\text{euf-cma}} &\leq \epsilon_{\text{XMSS}}^{\text{euf-rma}} + \epsilon_{\text{prf}}^{\text{ms}} + \epsilon_{\text{tcr}}^{\text{ro}}. \\ \epsilon_{\text{XMSS}}^{\text{euf-rma}} &\leq \epsilon_{\text{prf}}^{\text{skexp}} + \epsilon_{\text{ots}}^{\text{wots}} + \epsilon_{\text{thf-tcr}}^{\text{pkco}} + \epsilon_{\text{thf-tcr}}^{\text{htrree}}. \\ \epsilon_{\text{ots}}^{\text{wots}} &\leq \epsilon_{\text{thf-tcr}}^{\text{cf}} + \epsilon_{\text{thf-ud}}^{\text{cf}} + \epsilon_{\text{thf-pre}}^{\text{cf}}.\end{aligned}$$

In the above, the first equation gives a bound on the standard notion of security for digital signatures, existential unforgeability against chosen message attacks (euf-cma), for XMSS in terms of an advantage against XMSS in a weaker attack model where signed messages are chosen at random ($\epsilon_{\text{XMSS}}^{\text{euf-rma}}$), a statistical (multi-target) collision bound in the ROM ($\epsilon_{\text{tcr}}^{\text{ro}}$), and an advantage against the PRF security of a function used to generate the randomizer for the computation of the message representative ($\epsilon_{\text{prf}}^{\text{ms}}$). Then, the second equation expresses the euf-rma bound in terms of an advantage against the underlying one-time signature ($\epsilon_{\text{ots}}^{\text{wots}}$), another PRF advantage for the function expanding WOTS⁺ signature keys, and two (multi-target) collision resistance advantages against the THF used in the hash tree structure ($\epsilon_{\text{thf-tcr}}^{\text{htrree}}$) and the compression of WOTS⁺ public keys ($\epsilon_{\text{thf-tcr}}^{\text{pkco}}$). Finally, the third equation expresses the advantage against WOTS⁺ in terms of several advantages against the underlying THF: (multi-target) collision resistance ($\epsilon_{\text{thf-tcr}}^{\text{cf}}$), undetectability ($\epsilon_{\text{thf-ud}}^{\text{cf}}$), and preimage resistance ($\epsilon_{\text{thf-pre}}^{\text{cf}}$).

At this abstraction level, all THF assumptions are expressed over the same collection, which in the formalization is called thfc . Here, a collection means a set of THFs indexed by their input sizes: for each input size, the collection contains a THF that maps some global parameters pp , a tweak tw , and an input x to a fixed sized output y . In the bounds above, pkco , htrree and (the THF underlying) WOTS⁺ refer to functions in thfc , each used for a

different, but fixed, input size. We note that the security proof holds for any instantiation of this THF collection over which the security assumptions hold. The same is true for instantiations of the PRFs.

OUR INSTANTIATION. We use the cloning mechanism of EASYCRYPT to obtain a refined version of the security proof with (more) concrete instantiations for various algorithms left abstract in the high-level security result. Not only does this include the THF and PRF components mentioned above, but also many of the data types used to represent the various values used by the construction, and distributions over these types. A few choices in the formalization of [BDG⁺23] created some difficulties at this point. First and foremost, the types of inputs and outputs of THF were hardwired to lists of booleans, which required the introduction of significant boilerplate code around the byte-oriented operators used in the RFC specification. This becomes evident in the next snippet of EASYCRYPT code that shows how we instantiate the THF collection.⁹

```

op thfc(i : int, ps : nbytes, ad : adrs, x : bool list) =
  if i = 8 * n then BytesToBits (f ps ad (BitsToBytes x))
  else if i = 8 * n * 2 then
    let ad = set_tree_height ad ((get_tree_height ad) - 1) in
    let x1 = take (8 * n) x in
    let xr = drop (8 * n) x in
    BytesToBits (rand_hash ps ad (BitsToBytes x1) (BitsToBytes xr))
  else if i = 8 * n * len then
    let wpk = (chunk n (BitsToBytes x)) in BytesToBits (ltree ps ad wpk)

```

The instantiation provides three alternatives for the relevant input sizes of the input boolean list x :

- Hashes computed in WOTS⁺ chains have input size $8 * n$, in which case the operator used is simply called f .
- Hashes computed for the hash-tree nodes have input size $8 * n * 2$, in which case the RFC-level operator `rand_hash` is used, again requiring a bit-to-byte conversion. (There is also a re-encoding of tree indices/addresses to match the semantics of the security proof tree operators.)
- WOTS⁺ public keys are of size $8 * n * len$, for which case the THF is instantiated with an operator that represents the `ltree` computation. This operator takes a byte-oriented representation, which explains the conversion before evaluation.

Note also that the byte-oriented outputs of all of the RFC-level operators must be again converted to boolean lists. Despite the cumbersomeness of this bookkeeping, the meaning of the instantiation is clear: we will be assuming that f , `rand_hash`, and `ltree` jointly form a secure THF collection for the three (distinct) input sizes required by XMSS.

The refinement of types and operators described above requires proving all properties that were axiomatized in the high-level security proof. This includes, for example, proving that the distributions defined in our RFC-like specification (such as those used to sample arrays of bytes uniformly at random) satisfy the required well-formedness properties.

The most significant of these is a property stating that, for any two distinct messages, their WOTS⁺ (base- w) encodings, including the checksum, are such that each encoding has at least one digit position in which it is strictly smaller than the corresponding digit of the other. This ‘cover-free’ property is essential for the security proof, as it underpins the validity of the reduction for WOTS⁺. In proving this, we identify a design choice in

⁹In this and other code snippets in this section, we omit some (sub-)type injection and valuation operators for readability. We use subtyping for fixed-size lists to leverage the EasyCrypt type system for enforcing correct usage of types. As a result, proofs must track sufficient context concerning the relevant sizes to argue bijectivity of encodings.

the RFC specification that unnecessarily restricts the possible parameter sets for which the security proof applies.

Concretely, the specification is secure only under a simple yet non-trivial condition on w and ℓ_2 : $8 \nmid \ell_2 \cdot \log_2 w$. In essence, the issue arises from the part of the RFC responsible for converting the checksum into its base- w representation. This conversion uses a bit-shift operation which, when the condition does not hold, discards the most significant base- w digit(s) of the checksum, thereby breaking the essential property of the encoding. Fortunately, the condition is satisfied by all parameter sets proposed in RFC 8391 [HBG⁺18] and NIST’s SP800-208 [CAD⁺20]; however, it may not for parameter sets considered in future extensions if it is not explicitly taken into consideration.¹⁰ We have reported this observation as an erratum on RFC 8391 [HBG⁺18].¹¹

Notably, this issue only became apparent in this instantiation, and in particular not in the high-level security proof, nor in the functional correctness proof relating the Jasmin implementation to the RFC-like specification. The reason is that the former abstracts away from low-level encoding details, while the latter proof is carried out with fixed parameters for which the result holds.

5.2 Proving Security of the RFC Specification

As mentioned in the introduction, the standard description of XMSS [HBG⁺18] transcribed to EASYCRYPT in Section 3 does not structurally match the one adopted in the security proof, which is parametric on several components. So, the first step in linking the security proof is to create a re-interpretation of the RFC specification in EASYCRYPT that 1) matches the structure of the specification considered in the security proof, and 2) is functionally equivalent to the original one. We call this specification `XMSS_RFC_ABS`. This is syntactically identical to the original one, with the exception that the message compression operation `H_msg` is factored out as a call to an oracle—in the security proof, this is modeled as a random oracle—and by introducing functional operators that replace the imperative code for several hashing operations:

```

op prf : nbytes -> nbytes -> nbytes.
op ltree : seed -> adrs -> wots_pk -> nbytes.
op rand_hash : seed -> adrs -> nbytes -> nbytes -> nbytes.

```

These simple syntactic changes are justified by a proof stating that the two specifications match when 1) the semantics of these operators matches the semantics of the corresponding imperative code, and 2) the oracle for message compression is instantiated with `H_msg`. This proof is straightforward, but we document it here to clarify that this sanity check has been done in the development. As a result, modulo the random oracle heuristic, the security result we obtain for this variant of the specification applies to the RFC specification, under the assumptions that the concrete operator `thfc` satisfies the relevant tweakable hash function family security notions described at the beginning of this section, and similarly for the operators that instantiate the pseudorandom functions (PRF) required by XMSS.

THE SECURITY REDUCTION. We obtain the claimed security result as follows. We first instantiate the generic EUF-CMA definitions available in EasyCrypt with `XMSS_RFC_ABS` in the ROM, giving us a type for EUF-CMA adversaries. To upper bound the success probability of an EUF-CMA forger against `XMSS_RFC_ABS`, we construct the reduction algorithm `B` shown in Figure 8. This algorithm is rather straightforward: given an EUF-CMA adversary `A` against `XMSS_RFC_ABS` and access to a random oracle `RO`,¹² `B` passes

¹⁰Interestingly, the implementations we examined deviate slightly from the RFC in their handling of the encoding, and can in fact be proven secure for all natural parameter choices.

¹¹Instantiations of WOTS⁺ in SPHINCS⁺, with $w = 256$, violate the condition. However, there, the specifications for the relevant algorithms already correctly address the issue we note here.

¹²Types considered by the random oracles are the same in both specifications as per our instantiation.

```

module B_Oracles(O : SOracle_CMA) : SOracle_CMA = {
  proc sign(m : W8.t list) : sig_t = {
    sig <@ 0.sign(m);
    return (f| sig_idx = W32.of_int sig.^2.^1 ; r = sig.^1;
           r_sig = (map BitsToBytes sig.^2.^2, (rev (map (BitsToBytes sig.^2.^3)))) |});
  }
}.

module (B(A : Adv_EUFCMA_RO) : Adv_EUFCMA_RO) (RO : RO.POracle, O : SOracle_CMA) = {
  proc forge(pk : pkXMSSTWRFC) : W8.t list * sigXMSSTW = {
    f <@ A(RO, B_Oracles(O)).forge(f| pk_oid = impl_oid;
    pk_root = BitsToBytes pk.^1; pk_pub_seed = pk.^2|});
    return (f.^1, (f.^2.^r, to_uint f.^2.^sig_idx,
    map BytesToBits f.^2.^r_sig.^1, rev (map BytesToBits f.^2.^r_sig.^2);
  }
}.

```

Figure 8: Attacker against the security specification used in the security proof in EASY-CRYPT. Sub-typing operators are omitted for readability. Here, $\cdot.^{\cdot}$ denotes access to tuple and record fields, which in this case are the various components from signatures obtained from the security game (in `sign`) or the adversary (in `forge`).

RO directly to A and simulates the signature oracle by encoding the output signatures into the correct data types. Similarly, when adversary A outputs a forgery, B encodes the forgery to match the data types of the security specs.

At the high level, the proof is uninteresting and, on paper, we could describe it as follows. Because we establish a bijection between the data types used by the two specifications, the view presented to A (by our reduction B) is a perfect simulation of the view it expects to observe in its own EUF-CMA game. Thus, whenever A produces a successful forgery against XMSSTWRFC_ABS, B is also able to produce a successful forgery against the security specification. The above statement, although true, hides two interesting caveats. First, the claim that B’s simulation is perfect relies on the fact that the key generation, signature, and verification algorithms from the security spec are functionally equivalent (modulo the data type conversions) to those from the RFC. Proving this claim turned out to be highly non-trivial, as we discuss in the last part of this section. Second, we cannot prove that B has exactly the same probability of forging as A has. In fact, for the proof to commute, we need to assume an additional check that is not specifically required by the RFC. We elaborate on this next.

The main subtlety arises from the way the security proof encodes the index of a signature, i.e., the identifier of the WOTS⁺ secret key used to sign the corresponding message. More specifically, the security specification uses a finite type with cardinality 2^h , which means that the forgery produced by our adversary B must include an index in this range. However, the RFC specification allows indices to be 32-bit numbers. Thus, technically, A could produce a (valid) forgery with an index that does not fall in the range $[0, 2^h - 1]$, whereas the security proof implicitly assumes that all such forgeries are invalid. This implies that we can justify the reduction only up to forgeries produced by A with an index in the range $[0, 2^h - 1]$. In effect, then, our proof applies to a variant of the RFC where this check is made explicitly. It is beyond the scope of this paper to extend the security proof to cover the more general case: our intention here is to preserve the existing proof as much as possible, further understand the challenges that arise when formalizing security and correctness separately, and to assess these challenges for the new setting of hash-based signatures. Nevertheless, all our results should extend naturally if the result from [BDG⁺23] is appropriately generalized.

SECURITY VIA A NON-TRIVIAL PROOF OF FUNCTIONAL CORRECTNESS. Based on the preceding, two types of choices or patterns seem to arise when devising a specification

```

type 'a bintree = [ Leaf of 'a | Node of 'a bintree & 'a bintree ].

op list2tree_pred ['a] (s : 'a list) (t : 'a bintree) =
  let e = height t in size s = 2 ^ e /\ fully_balanced t
  /\ (forall idx, 0 <= idx < size s => extr_valllf t (rev (int2bs e idx)) = onth s idx).

op list2tree ['a] (s : 'a list) : 'a bintree =
  if (exists e, 0 <= e /\ size s = 2 ^ e) then choiceb (list2tree_pred s) witness
  else witness.

op val_bt_trh (bt : dgstblock bintree) (ps : pseed)
  (ad : adrs) (hidx : int) (bidx : int) : dgstblock =
  with bt = Leaf x => x
  with bt = Node l r =>
    trh ps (set_thtbidx ad hidx bidx)
    (val (val_bt_trh l ps ad (hidx - 1) (2 * bidx)) ++
     val (val_bt_trh r ps ad (hidx - 1) (2 * bidx + 1))).

```

Figure 9: Functional definition of TREEHASH in EASYCRYPT.

over which to carry out a security proof: the first is whether to provide concrete types and operators, or simply declare them and axiomatize the properties required for the security proof; the second is whether (and how) to provide a functional description of (parts of) the considered cryptographic algorithms, or provide these imperatively.

For the former, when considering only the security proof, the kind of type or operator definition (i.e., concrete or abstract) often directly follows the original theoretical (pen-and-paper) security proof that establishes the claim. This makes the meaning of the formal verification result clearer and, one could argue, more useful. However, one thing we discover in this work is that there is a loss in generality that comes from making types and operators concrete. In particular, this is the case for the input and output types of THFs (boolean lists), as well as the type for indices (finite range); these specification decisions can have a large impact on the result’s reusability.

In the remainder, we focus on the impact of choices related to the second pattern above: the description of operators and algorithms. EASYCRYPT’s program logics offer natural reasoning for code-based security proofs, where security games are imperative programs, oracles are procedure calls, and random samplings are a special kind of imperative statement. This means that for deterministic (parts of) cryptographic algorithms, such as those for XMSS, one gets some flexibility. A prime example is the functional definition of the TREEHASH computation used in the security specification of [BDG⁺23] (Figure 9).

Intuitively, the formalization in Figure 9 captures TREEHASH as a three-step process: First, one defines a list with all the tree leaves; then, one uses the `list2tree` operator to obtain a binary tree of type `bintree`, where only the values in the leaves are defined (this only works if we have a full tree, otherwise the result is set to some opaque constant value `witness`); and, finally, one applies the `val_bt_trh` operator to recursively fill the non-leaf nodes with the hash (`val_bt_trh`) of their descendants. The main motivation for this definitional style is the ability to 1) apply induction principles over the tree structure when conducting various proofs, and 2) easily deal with random samplings associated with leaf generation, since they are all performed upfront.

Interestingly, this induction principle then also drives the style of imperative description of TREEHASH encountered in the various steps of the proof from [BDG⁺23]. These imperative descriptions follow the three-step process above and, when computing the hash values for the tree, do this level by level in order to easily relate the functional specification with the intermediate imperative states. Clearly such computations are sufficient for a security proof, but no practical implementation would follow this pattern. Indeed, the description of TREEHASH in Section 3 is optimized for space by generating leaves on the

```

lemma tree_hash_correct _ps _ss _lstart _sth :
phoare [ TreeHash.treehash :
    arg = (_ps, _ss, _lstart, _sth, zero_address)
    /\ 0 <= _sth <= h /\ 0 <= _lstart <= 2h - 2_sth /\ 2_sth %| _lstart
    ==> BytesToBits res =
    val_bt_trh (list2tree (map (leafnode_from_idx _ss _ps (adr2ads zero_address))
    (range _lstart (_lstart + 2_sth)))) _ps
    (set_typeidx (adr2ads zero_address) trhtype) _sth (_lstart %/ 2(_sth + 1)) ] = 1%r.

```

Figure 10: EASYCRYPT lemma for the correctness of TREEHASH.

fly and using a stack to keep track of intermediate results. Although more aggressive optimizations of TREEHASH can be found in practice, already this ‘small’ abstraction gap turned out to be the most challenging part of our proof. On the positive side, as shown in Figure 2 and discussed in Section 3, TREEHASH encompasses essentially all of the complex computations performed by the key generation and signing algorithms, which means that a large part of the proof effort for these optimized algorithms can be amortized. Investigating proofs for the even more aggressively optimized implementations is interesting future work. We conclude this section with a description of our functional correctness proofs relating the two specifications involved in the security proof.

TREEHASH. In order to capture all uses of the TREEHASH algorithm described in Section 3, we prove a general functional correctness lemma (using PHL) that captures total correctness, see Figure 10. For deterministic and stateless programs, such statements exactly correspond to the statement that, for every input satisfying the pre-condition, the program 1) terminates and 2) returns a value satisfying the post-condition.

In this case, several parameters are involved (listed before the initial colon). First, parameters `_ps` and `_ss` represent seeds used by the algorithm to expand the randomness needed to generate leaf nodes; indeed, these parameters allow the lemma to be applied for any externally generated seeds. More interestingly, parameters `_lstart` and `_sth` identify the subtree whose root is computed by respectively denoting the index of the leftmost leaf of the subtree and the height of the subtree. The precondition then restricts the range of values for which the result should hold: 1) the height `_sth` must be between 0 (when “computing” a leaf) and h (when computing the root), and 2) the index `_lstart` must be consistent with one of the subtrees of height `_sth` (the `%|` operator represent integer divisibility). The post-condition, unsurprisingly, states that the result `res` contains the value defined by the functional operator `val_bt_trh` for the same subtree (the last parameter is the height of the root node of the subtree).

We divide the proof into two steps, taking advantage of EasyCrypt’s expressive logics. We first define a version of the TREEHASH algorithm that abstracts away all the unnecessary low-level details that are present in the RFC specification. This is shown in Figure 11. The algorithm is given access to the full list of tree leaves, and uses a list to implement the stack. Furthermore, it works over an abstract type `value` for nodes (and leaves), additionally abstracting away all irrelevant hashing-related and address-related details. Listed below in the same figure is the correctness lemma we establish for this algorithm. Here, `reduce_tree` is, again, a functional operator that is defined recursively, analogously to the one in Figure 9, but again abstracting away the details inherent to the security proof. In this way, we can carry out the correctness proof for TREEHASH without worrying about mismatches between the security proof logic and the RFC specification logic other than the way in which the TreeHash computation is expressed. Given its generality, see this result as being of independent interest for future work in analysing algorithmic optimizations for hash-tree-based cryptographic constructions.

The crux of this proof are two invariants that express the state of the stack in each iteration of the outer loop and each iteration of the inner loop, respectively. These

```

proc th(pseed : pseed, leaves : value list, root : haddress) : value = {
  stack <- []; idx <- 0;
  offset <- root.`index * 2^(root.`level);
  while (idx < 2^(root.`level)) {
    focus <- (leaves.[offset + idx], 0);
    while (stack <> [] /\ stack.[0].`2 = focus.`2) {
      top <- stack.[0].`1; stack <- behead stack;
      addr <- {| level = focus.`2; index = (offset + idx) %/ (2^(focus.`2 + 1)) |};
      focus <- (hash pseed addr top focus.`1, focus.`2 + 1); }
    stack <- focus :: stack; idx <- idx + 1; }
  return stack.[0].`1; }

lemma treehash_pcorrect (_pseed : pseed) (_leaves : value list) (_root : haddress) :
  size _leaves = 2^h => valid_haddress _root
=> phoare[TreeHash.th : arg = (_pseed, _leaves, _root) ==>
  res = reduce_tree _pseed _leaves _root ] = 1%r.

```

Figure 11: Simplified TREEHASH in EASYCRYPT.

```

op stackrel (root : haddress) (pseed : pseed) (leaves : value list)
  (idx : int) (stk : stack) =
  let s = int2bs (root.`level + 1) idx in
  (ones s = map (fun (stk1 : stack1) => stk1.`2) stk) /\
  (forall stk1, stk1 \in stk => stk1.`1 = let addr = {| level = stk1.`2;
    index = 2^(root.`level - stk1.`2) * root.`index + bs2int (false :: drop (stk1.`2 + 1) s);
  |} in reduce_tree pseed leaves addr).

op eqvred (off : haddress) (pseed : pseed) (s1 s2 : stack) =
  forall (P : stack -> stack -> bool),
    (forall s, P s s)
=> (forall s2 s1 s3, P s1 s2 => P s2 s3 => P s1 s3)
=> (forall s v1 v2 i,
  let addr = {| level = i; index = (haddr2off off + revones (unzip2 s)) %/ 2^(i+1) |} in
  P ((v2, i) :: (v1, i) :: s) ((hash pseed addr v1 v2, i + 1) :: s)) => P s1 s2.

```

Figure 12: Invariants for TREEHASH correctness.

invariants are written in terms of the operators shown in Figure 12. The first operator `stackrel` describes the value of the stack in terms of the index of the leaf node that is being processed, requiring an intricate interpretation of the bit decomposition of the index value. The second operator `eqvred` describes what it means for one stack to result from a reduction step (i.e. hashing two tree nodes) applied to a previous stack. Proving that these invariants hold throughout the execution of the two loops requires a number of auxiliary results about the behavior of paths and their prefixes, including when the index of a leaf is incremented. The most intricate lemmas show that the exit condition of the inner loop (when we encounter two nodes at the top of the stack at different heights) exactly happens when the invariant for the inner loop (based on the hamming weight of the path to the next leaf) is satisfied. Other lemmas guarantee that accesses to all structures are within bounds, i.e., that the stack space is not exceeded, as mentioned in the previous section.

We then proceed by proving a functional equivalence between the RFC specification of TREEHASH and our simplified TREEHASH algorithm above when (the latter is) appropriately instantiated. That is, we instantiate the abstract types and operators used by `th` (our simplified TREEHASH) with the concrete ones used by `TreeHash.treehash` (the RFC’s TREEHASH). The equivalence lemma then states that both algorithms produce the same result if all inputs satisfy the necessary structural restrictions described above. The proof of this lemma is mostly boilerplate, relating the state of the two algorithms as they progress in lockstep. Finally, EasyCrypt’s ambient logic allows us to compose this equivalence with the previously established correctness of our simplified TREEHASH

algorithm to derive that the RFC and abstract specification compute the same result.

PROVING XMSS KEY GENERATION, SIGNING AND VERIFICATION. After proving the correctness of `TREEHASH` and (the instantiation of) the other functional operators, the equivalence proofs for the procedures as a whole, although laborious, follow a more structural pattern based on the function call tree. For key generation, the equivalence follows directly from the fact that the security specification applies the functional `TREEHASH` operator on a precomputed list of leaves for the entire tree. This then commutes with the result of the imperative `TREEHASH` computation carried out by the RFC, as guaranteed by the correctness result we established before. For signing and verification, though, we must prove the preservation of additional invariants. Nevertheless, they are in the same vein as the ones before, relating values computed through functional operators to the (intermediate) values computed by imperative code. In the proof of the signing procedure, the main additional invariant relates the functional and imperative computation of the authentication path. Here, the imperative computation of the authentication path is done by calling `TREEHASH` for each level in the tree; the functional computation of the authentication path is expressed as follows:

```

op cons_ap_trh_gen (bt : dgstblock bintree) (bs : bool list) (ps : pseed)
  (ad : adrs) (hidx : int) (bidx : int) : dgstblock list =
  with bt = Leaf _, bs = [] => []
  with bt = Leaf _, bs = _ :: _ => witness
  with bt = Node _ _, bs = [] => witness
  with bt = Node l r, bs = b :: bs' =>
    (val_bt_trh (if b then l else r) ps ad (hidx - 1) (if b then 2 * bidx else 2 * bidx + 1))
    :: cons_ap_trh_gen
      (if b then r else l) bs' ps ad (hidx - 1) (if b then 2 * bidx + 1 else 2 * bidx).

```

Although the proof requires some effort due to the significant boiler-plate proof requirements arising from the data type conversions, the state of the iterative computation is not too difficult to express in terms of the functional values of the authentication path. From this, the desired equality between (the results of) the imperative and functional computations follows. Lastly, the verification procedure follows a largely similar pattern, but does not require `TREEHASH`. In essence, verification produces a leaf and iteratively hashes it together with the given authentication path (while updating the addresses), ultimately producing a candidate root node. This process does not rely on `TREEHASH`. In the imperative setting, it is implemented as a straightforward for loop. In the functional setting, the same logic is captured by the following operator:

```

op val_ap_trh_gen ap bs leaf ps ad hidx bidx =
  with ap = [], bs = [] => leaf
  with ap = [], bs = _ :: _ => witness
  with ap = _ :: _, bs = [] => witness
  with ap = x :: ap', bs = b :: bs' =>
    trh ps (set_thtbidx ad hidx bidx)
    (if b
      then val x ++ val (val_ap_trh_gen ap' bs' leaf ps ad (hidx - 1) (2 * bidx + 1))
      else val (val_ap_trh_gen ap' bs' leaf ps ad (hidx - 1) (2 * bidx)) ++ val x).

```

Once again, although somewhat laborious, the necessary invariants are not too difficult to express and maintain, providing a relatively clear and direct path to the desired result.

6 Conclusion

We have presented the first, to the best of our knowledge, formally verified implementation of a hash-based signature, for which we establish a machine-checked connection to a provable security result.

This effort served to highlight an improvement to the RFC text that makes it more robust to future extensions that include additional parameter sets, as discussed in Section 5. This has been reported as an erratum on RFC 8391 [HBG⁺18].

More generally, we find that hash-based signatures pose a different set of challenges for formal verification than algebraically-flavored algorithms such as ML-KEM and ML-DSA. We believe that these challenges will be even more evident in more complex constructions based solely on symmetric primitives such as SPHINCS+ [HBD⁺20] and MPC-in-the-Head derivatives such as PICNIC3 [ZCD⁺20] and syndrome-decoding-in-the-head [AFG⁺24]. For future work there are several interesting directions, such as tackling the more complex algorithms described above, and exploring the correctness of aggressive optimizations of the authentication-path algorithms. Of these, those that require reasoning about pre-computation of partial results of multiple TREEHASH computations to guarantee a small variance in the execution of signing operations seem particularly challenging.

References

- [ABB⁺17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. Jasmin: High-assurance and high-speed cryptography. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1807–1823. ACM Press, October / November 2017.
- [ABB⁺19] José Bacelar Almeida, Cecile Baritel-Ruet, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Alley Stoughton, and Pierre-Yves Strub. Machine-checked proofs for cryptographic standards: Indifferentiability of sponge and secure high-assurance implementations of SHA-3. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1607–1622. ACM Press, November 2019.
- [ABB⁺20] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub. The last mile: High-assurance and high-speed cryptographic implementations. In *2020 IEEE Symposium on Security and Privacy*, pages 965–982. IEEE Computer Society Press, May 2020.
- [ABB⁺23] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, Antoine Séré, and Pierre-Yves Strub. Formally verifying Kyber episode IV: Implementation correctness. *IACR TCHES*, 2023(3):164–193, 2023.
- [ABBD13] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, and François Dupressoir. Certified computer-aided cryptography: efficient provably secure machine code from high-level implementations. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 1217–1230. ACM Press, November 2013.
- [ABBD16] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, and François Dupressoir. Verifiable side-channel security of cryptographic implementations: Constant-time MEE-CBC. In Thomas Peyrin, editor, *FSE 2016*, volume 9783 of *LNCS*, pages 163–184. Springer, Berlin, Heidelberg, March 2016.
- [AFG⁺24] Carlos Aguilar-Melchor, Thibault Feneuil, Nicolas Gama, Shay Gueron, James Howe, David Joseph, Antoine Joux, Edoardo Persichetti, Tovohery H. Randrianarisoa, Matthieu Rivain, and Dongze Yue. SDitH — Syndrome Decoding in the Head. Technical report, National Institute of Standards and Technology, 2024. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>.
- [AOB⁺24] José Bacelar Almeida, Santiago Arranz Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Cameron Low, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, and Pierre-Yves Strub. Formally verifying kyber - episode V: Machine-checked IND-CCA security and correctness of ML-KEM in EasyCrypt. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 384–421. Springer, Cham, August 2024.

- [BBB⁺21] Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. SoK: Computer-aided cryptography. In *2021 IEEE Symposium on Security and Privacy*, pages 777–795. IEEE Computer Society Press, May 2021.
- [BDG⁺13] Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. Easycrypt: A tutorial. In Alessandro Aldini, Javier López, and Fabio Martinelli, editors, *Foundations of Security Analysis and Design VII - FOSAD 2012/2013 Tutorial Lectures*, volume 8604 of *Lecture Notes in Computer Science*, pages 146–166. Springer, 2013.
- [BDG⁺23] Manuel Barbosa, François Dupressoir, Benjamin Grégoire, Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. Machine-checked security for XMSS as in RFC 8391 and SPHINCS⁺. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 421–454. Springer, Cham, August 2023.
- [BDH11] Johannes A. Buchmann, Erik Dahmen, and Andreas Hülsing. XMSS - A practical forward secure signature scheme based on minimal security assumptions. In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011*, pages 117–129. Springer, Berlin, Heidelberg, November / December 2011.
- [BDH⁺24] Manuel Barbosa, François Dupressoir, Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. A tight security proof for SPHINCS⁺, formally verified. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 35–67. Springer, Singapore, December 2024.
- [BGLP21] Gilles Barthe, Benjamin Grégoire, Vincent Laporte, and Swarn Priya. Structured leakage and applications to cryptographic constant-time and cost. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 462–476. ACM Press, November 2021.
- [CAD⁺20] David Cooper, Daniel Apon, Quynh Dang, Michael Davidson, Morris Dworkin, and Carl Miller. Recommendation for stateful hash-based signature schemes, 2020-10-29 00:10:00 2020.
- [DPW11] Jean Paul Degabriele, Kenneth G. Paterson, and Gaven J. Watson. Provable security in the real world. *IEEE Secur. Priv.*, 9(3):33–41, 2011.
- [HBD⁺20] Andreas Hülsing, Daniel J. Bernstein, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, Jean-Philippe Aumasson, Bas Westerbaan, and Ward Beullens. SPHINCS⁺. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [HBG⁺18] A. Huelsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen. Rfc 8391: Xms: extended merkle signature scheme, 2018.
- [HMS22] Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. Formal verification of Saber’s public-key encryption scheme in EasyCrypt. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 622–653. Springer, Cham, August 2022.

- [Hül17] Andreas Hülsing. WOTS+ – shorter signatures for hash-based signature schemes. Cryptology ePrint Archive, Report 2017/965, 2017.
- [MCF19] D. McGrew, M. Curcio, and S. Fluhrer. Rfc 8554: Leighton-micali hash-based signatures, 2019.
- [Nat24] National Institute of Standards and Technology. FIPS PUB 205 – SLH-DSA: Stateless hash-based digital signature standard, 2024. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf>.
- [OBG⁺24] Santiago Arranz Olmos, Gilles Barthe, Ruben Gonzalez, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, and Peter Schwabe. High-assurance zeroization. *IACR TCHES*, 2024(1):375–397, 2024.
- [SBG⁺22] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, and Swarn Priya. Enforcing fine-grained constant-time policies. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 83–96. ACM Press, November 2022.
- [SBG⁺23] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean. Typing high-speed cryptography against spectre v1. In *2023 IEEE Symposium on Security and Privacy*, pages 1094–1111. IEEE Computer Society Press, May 2023.
- [ZCD⁺20] Greg Zaverucha, Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, Jonathan Katz, Xiao Wang, Vladimir Kolesnikov, and Daniel Kales. Picnic. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.